MuJoCo steps

A practical workflow for your actual Wall-E assembly

First milestone: the assembled robot appears in MuJoCo at the correct scale, with correctly rotating wheels, credible mass distribution, and plausible ground contact. Motor characterization and reinforcement learning follow after the mechanical checks pass.

STEP files supply geometry. A moving simulation also needs body relationships, joint axes, masses, inertia, contacts, and actuators. A correct-looking animation is not evidence of accurate physical behavior.

1. Prepare the Onshape assembly

• Place components exactly as installed, including batteries, PCB, motor housings, brackets, and fasteners. Confirm the battery location against the physical robot.

• Treat everything rigidly attached to the chassis as one moving body. Keep the left and right rotating wheel assemblies as two separate bodies. Include hubs or rotating shaft attachments where relevant.

• Use fastened mates for rigid attachments and one revolute mate per wheel, aligned with the actual motor shaft. The initial model can represent gearbox effects without simulating internal gears.

Body structure: floating chassis → left wheel hinge and right wheel hinge. The chassis must ultimately be free to translate and rotate relative to the ground.

• Choose X forward, Y left, Z up. Use the midpoint between wheel centers as the robot reference origin. Record the MPU6050 location and mounting orientation in this frame.

Checkpoint: Only the two intended wheel rotations remain within the assembly; the installed geometry and component positions match the physical robot.

Current hardware context

ESP32-S3-WROOM-1, MPU6050, and two TT motors without encoders are recorded. The schematic includes a TB6612FNG module; confirm its physical marking. The supplied component subtotal is 186 g, excluding batteries, screws, and unlisted parts. It is not the assembled mass.

Measure and export

2. Establish geometry and mass properties

Parameter	Obtain / check
Wheel radius and width	CAD dimensions; verify real wheels and loaded rolling radius.
Wheel-center spacing	CAD distance between wheel centers; physical cross-check.
Complete assembled mass	Weigh the complete robot with installed batteries.
Chassis and wheel masses	Separate chassis from rotating wheel assemblies; avoid double counting.
Center of mass	CAD estimate using credible component masses; physical check with battery installed.
Inertia tensors	CAD estimates for chassis and each rotating wheel, with their reference frames.
IMU mounting	Position, orientation, and axis mapping relative to the chassis.

Inertia measures resistance to rotation. The same mass placed higher changes balancing dynamics. Assign appropriate materials and measured mass overrides in Onshape, then inspect Mass Properties. Simplified motors and batteries may still have approximate internal mass distributions.

Record kilograms, meters, and kg·m² in the model; record measurement uncertainty and distinguish CAD estimates from physical measurements. Inertias must be expressed about the appropriate center of mass and in a known frame.

3. Export the assembly

• Preferred route: right-click the Onshape Assembly tab → Export → URDF, if available. Keep the complete package, including meshes. Save a version of the assembly for traceability.

• Load and inspect the URDF in MuJoCo, then save it as MJCF, MuJoCo's native XML format. Check joint axes, transforms, masses, and inertias; add simulation-specific settings in MJCF.

• Fallback: export individual binary STL meshes in meters plus a full assembly STEP reference. Preserve or record part placements so the assembly can be reconstructed in MJCF.

Checkpoint: Geometry has a known scale and reference frame; rotating wheels remain distinct; mass data and unknowns are recorded before physical-model implementation.

Build the first simulation

4. Create and check the MuJoCo scene

• Use a free joint for the chassis and a hinge joint for each wheel. Define explicit mass, center of mass, and inertia for each moving body using the measured and documented parameters.

• Add a flat floor. Use CAD meshes for appearance and simple collision shapes for the chassis and wheels. Fine details such as PCB traces generally do not need collision geometry.

• Check that visual geometry does not accidentally add mass or unwanted contacts. Ensure wheel contacts and chassis ground clearance match the assembly.

• Inspect the stationary model first: scale, wheel centers, shaft axes, mesh orientation, and initial placement. Avoid starting with wheels penetrating the floor.

• With motor commands at zero, start from a small tilt and observe passive falling. Compare with safe physical observations where available; gearbox drag can affect the real zero-command behavior.

• Compare trajectories using smaller physics timesteps. Large changes indicate unresolved numerical or contact issues. Record the timestep, solver settings, and model revision.

Checkpoint: The model compiles; wheels rotate about the correct axes; contact and passive motion are plausible; timestep checks do not reveal major instability.

5. Add characterized motor behavior

A wheel torque command in MuJoCo is not automatically an ESP32 PWM command. Measure the motor/driver response before selecting realistic limits or a PWM-to-torque model.

• Confirm driver marking, supply voltage range, motor polarity, command dead zone, left/right asymmetry, response delay, and speed/load behavior through supported tests.

• Use credible torque limits and account for battery voltage, motor saturation, lag, gearbox losses, and tire friction. Unloaded speed alone does not establish available torque.

• Test equal forward commands, equal reverse commands, and differential commands. Verify the physical command signs and expected robot motion.

Checkpoint: Motor parameters have measurement evidence or clearly documented uncertainty; simple commanded responses behave plausibly against that evidence.

Keep development incremental

CAD preparation and physical measurements can proceed together. Do not substitute generic TT motor specifications or incomplete component totals for this robot’s measured parameters. Simulation results remain simulation evidence until checked against hardware.

Connect sensing and learning

6. Add sensors, a baseline, and the RL interface

• Place virtual IMU sensing at the physical MPU6050 location. Model the pitch estimation pipeline, gyro bias/noise, sample timing, and delay based on measurements.

• MuJoCo exposes wheel position and speed for diagnostics, but the physical robot has no encoders. Deployable policy inputs must use available measurements or a validated estimator.

• Test a simple balancing controller as a diagnostic baseline after the physics checks pass. Compare pitch response, drift, control effort, and failure behavior.

• Wrap the simulation in Gymnasium: reset the robot, apply a motor command, advance physics, return sensor-like observations and reward, and report fall/time-limit endings. Verify the interface with check_env.

• Then train PPO with reproducible seeds, configurations, dependency versions, checkpoints, and independent evaluations. Add measured noise, latency, and parameter uncertainty; evaluate unseen conditions.

Connection to CartPole: observe → choose action → advance physics → receive reward. Here the observation and command mapping must reproduce what the robot can actually sense and execute. Keep training, simulation, deployment, and independent safety cutoffs separate.

Checkpoint: The environment passes interface checks and physical sanity checks before robot PPO training begins.

Immediate handoff checklist

• Complete the assembly structure and check the URDF export option.

• Supply the exported package’s local path and retain the assembly STEP reference.

• Supply assembled mass with batteries, wheel dimensions, and wheel-center spacing. Include screw/battery counts and any missing component masses.

• Supply CAD center-of-mass and inertia estimates with units/reference frames, plus IMU mounting information. Mark values not yet measured as unknown.

Project files to maintain

configs/robot.yaml: measured parameters. docs/HARDWARE.md: confirmed facts and unknowns. simulation/models/: future MJCF and meshes. simulation/environments/: future Gymnasium wrapper. docs/PLAN.md and docs/PROJECT_MAP.md: milestone and layout changes.

Official references

Onshape: exporting files (including Assembly URDF and STL options)

Onshape: Mass Properties tool

MuJoCo: modeling and URDF-to-MJCF workflow

MuJoCo: XML model reference